# Governed Memory with Tamper-Evident Audit: A Local-First Schema-Grounded Memory System for Multi-Client AI Agents

Anonymous

**Abstract**

AI agent memory systems must balance three requirements that existing architectures address only in isolation: (1) temporal correctness when facts are corrected or superseded, (2) tamper-evident auditability of memory operations, and (3) multi-client deployment on shared hardware. We present MemTether, a local-first agent memory system that integrates all three in a single architecture. MemTether stores memories as typed rows in a local SQLite database, maintains bi-temporal validity (valid time vs. system-known time) with supersession chains, provides a hash-chained tamper-evident audit log with browser-verifiable evidence export, and enables multiple AI clients on the same machine to share one physical database via file-level pointers. In 26 days of production deployment across 15 agent clients, MemTether processed over 31K recorded operations (2,346 writes tracked in the audit log plus 28,807 tracked reads) with zero data loss and 347 supersession events. On a controlled tamper-detection experiment over 2,346 audit entries, the hash chain detected 100% of edit and deletion attacks (5/5 scenarios) while correctly accepting unmodified logs. We additionally report a schema-grounded SQL retrieval path that achieves 100% recall on entity-aggregation queries in controlled settings, and discuss its current limitation on English-language benchmark data. We release the system as an Apache-2.0 open-source project with 488 passing tests and a 9-job CI matrix across three operating systems.

## 1 Introduction

AI agents that interact with users over extended periods accumulate a growing body of contextual knowledge—user preferences, system configurations, past decisions, and learned procedures. Memory systems such as mem0 (Chhikara et al., 2025), Zep (Rasmussen et al., 2025), and Letta (Packwood et al., 2025) store this knowledge and retrieve relevant portions to augment the agent's context. However, production deployment of such systems raises three requirements that no single existing system fully addresses:

1. **Temporal correctness.** When an agent corrects a previously stored fact, the old value must not be silently deleted or silently retained; it must be tracked as superseded with explicit validity boundaries. Without this, the system either loses history (deletion) or serves stale facts (unbounded retention). MemStrata (Yadav, 2026) has recently addressed this with bi-temporal ledgers; we independently implement the same principle.

2. **Tamper-evident auditability.** For compliance (EU AI Act Art. 12 (EU AI Act, 2024)), debugging, and trust, memory operations should be verifiable. No prior agent memory system, to our knowledge, provides a hash-chained audit log with browser-verifiable evidence export.

3. **Multi-client deployment.** Multiple AI clients (chat assistants, coding agents, IDE plugins) on the same machine should share memory without cloud synchronization or per-client duplication. Existing systems either require a hosted server (mem0, Zep) or are single-process (Letta, LangMem).

We present MemTether, an agent memory system designed around these three requirements. MemTether's core architecture is a single SQLite database shared across 23 client adapters via NTFS junctions. Every memory write is recorded in a hash-chained audit log; every 10 operations produce a SHA-256 anchor over

the log. Any subsequent edit or deletion breaks the chain and is detected. Corrections create supersession chains with bi-temporal timestamps (valid time and system-known time are tracked independently). A schema-grounded SQL retrieval path routes structured queries to deterministic lookups, though we show that its current implementation is effective on Chinese-language data: a controlled synthetic benchmark of 10 entities $\times$ 3 attributes $\times$ multiple sessions (25 facts per entity) achieves 100% recall via SQL versus 62% for embedding search; on 1,662 production Chinese memories, the predicate extractor yields (entity, attribute) pairs for 82.6% of fact-type entries. The SQL path has not yet been evaluated on English-language benchmarks..

Our contributions:

- We design and deploy a production agent memory system with bi-temporal supersession, hash-chained audit, and file-level multi-client sharing in a single architecture (26 days, 31K recorded operations, 15 agents, 0 data loss).
- We provide a controlled tamper-detection evaluation showing that the hash chain detects 100% of edit and deletion attacks (5/5 scenarios) on 2,346 audit entries, with explicit discussion of the unanchored-tail limitation.
- We report a negative result: our schema-grounded SQL path, while effective on Chinese-language synthetic data (100% recall), does not improve LongMemEval (Wu et al., 2024) accuracy on English-language data due to the lack of a general English predicate extractor. We identify this as a concrete open problem.

# 2 Background and Related Work

## 2.1 Agent Memory Systems

mem0 (Chhikara et al., 2025) provides a hosted memory platform with LLM-based fact extraction and vector retrieval. Zep (Rasmussen et al., 2025) offers a temporal knowledge graph. Letta (Packwood et al., 2025) manages hierarchical memory. LangMem (Liu et al., 2025) integrates memory into LangGraph. Engram (Wang et al., 2026) achieves 83.6% on LongMemEval-S with a bi-temporal model but lacks governance features. G-Memory (Kim et al., 2025) proposes a three-layer graph hierarchy for multi-agent systems. None provide hash-chained audit trails or file-level multi-client deployment.

## 2.2 Schema-Grounded Memory

xmemory (Petrov et al., 2026) proposes schema-grounded memory with iterative schema-aware extraction, achieving 97.10% F1 on end-to-end memory benchmarks. Memanto (Abtahi et al., 2026) presents typed semantic memory with information-theoretic retrieval, achieving 89.8% on LongMemEval. MemStrata (Yadav, 2026) maintains temporal validity with deterministic supersession rules in a bi-temporal ledger, reaching 95% on LongMemEval. These systems demonstrate that schema-grounded, deterministic retrieval is a promising direction. However, none provide tamper-evident audit, multi-client file-level sharing, or human-adjudication governance.

## 2.3 Tamper-Evident Logging

A2A-ForensicTrace (A2A-ForensicTrace, 2026) provides offline verification of tamper-evident runtime evidence. Proof-of-Execution Memory (PoEM, 2026) defends against forged-reasoning attacks. MemMark (MemMark, 2026) applies watermarking to agent memory. These approaches address adjacent problems; none implement a hash-chained memory audit log with human-verifiable evidence export in the agent memory layer.

# 3 System Design

## 3.1 Typed Memory Schema

MemTether supports 23 pre-defined client adapters (Claude Desktop, Claude Code, Cursor, Windsurf, VS Code, Zed, Cline, Roo Code, Kilo Code, JetBrains IDEs, Neovim, Amazon Q, Gemini CLI, Cody, Continue, Codex, WorkBuddy CN/Intl, CodeBuddy, ZCode, dsh, OpenClaw, and a neutral fallback). Of these, 15 distinct agent clients have actually written to the production database. MemTether stores memories as rows in a local SQLite database:

```
CREATE TABLE facts (
    uid TEXT UNIQUE, type TEXT, subject TEXT,
    content TEXT, status TEXT, source TEXT,
    valid_from TEXT, valid_to TEXT,
    recorded_at TEXT, invalidated_at TEXT,
    superseded_by TEXT, q_value REAL,
    use_count INTEGER, ...
);
CREATE TABLE audit_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    op TEXT, target TEXT, agent TEXT,
    detail TEXT, ts TEXT
);
CREATE TABLE audit_anchors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    last_log_id INTEGER, chain_hash TEXT,
    entries_hashed INTEGER, ts TEXT
);
CREATE TABLE supersessions (
    old_uid TEXT, new_uid TEXT, reason TEXT, by_agent TEXT, ts TEXT
);
```

The `source` column is enforced: every write must carry a source identifier that is registered in a per-deployment `agents.json`. Unregistered sources are rejected.

**Read tracking.** Read operations are tracked via the `use_count` column on each fact row: when a fact is returned by a retrieval query and the caller acknowledges using it, the `use_count` is incremented. Over 26 days of production, the cumulative sum of `use_count` across active facts is 28,807. Combined with 2,346 write operations recorded in `audit_log`, this yields the 31K+ total operations reported in Table 3. We note that this counter tracks *acknowledged* retrievals only; other read paths (e.g., ad-hoc SQLite queries bypassing the API) are not counted.

## 3.2 Bi-Temporal Model

Each fact carries two independent time axes: `valid_from`/`valid_to` (when the fact was true in the real world) and `recorded_at`/`invalidated_at` (when the system learned or disbelieved it). When an agent corrects a fact, the old row is marked `superseded` with `valid_to` set to the new fact's `valid_from`, creating an immutable supersession chain. The full edit history of any memory is reconstructable by walking the chain.

## 3.3 Memory Court: Hash-Chained Audit

Every memory operation (remember, correct, retire, conflict verdict, q-value update) is appended to `audit_log`. Every 10 operations, a SHA-256 anchor is computed:

$$h_i = \text{SHA256}(h_{i-1} \| \text{row}_j \| \text{row}_{j+1} \| \cdots \| \text{row}_{j+9})$$

where $h_0$ is a genesis constant and $\text{row}_j$ is the canonical text representation of the $j$-th audit row. Anchors are stored in `audit_anchors`. Any edit to an anchored row breaks the recomputation of the covering anchor; any deletion of anchored rows causes a row-count mismatch. Evidence can be exported as a browser-verifiable ZIP: an independent verifier recomputes the chain in JavaScript without trusting the exporter. In production, a `memtether court` CLI provides: (1) `verify` – recompute the full chain and report mismatches; (2) `export` – bundle the audit log, anchors, and a self-contained HTML verifier into a ZIP that can be opened in any browser without installing MemTether; (3) `conflicts` – list candidate contradictions between same-entity facts for human review, with an `accept/reject` verdict that is itself recorded in the audit log. Conflicts detected by rule-based heuristics are surfaced as *candidates*, not automatic decisions; a human or designated agent must review each one.

### 3.4 File-Level Multi-Client Sharing

On Windows, the SQLite database is stored in a user-designated directory and shared via NTFS junctions to each AI client's data path. On macOS and Linux, symbolic links serve the same role. The file-level pointer mechanism is OS-specific but conceptually identical: the database file lives in exactly one location, and every client resolves to it through a filesystem-level alias. The 9-job CI matrix covers Ubuntu, Windows, and macOS for the core library, but the file-pointer mechanism itself is validated primarily on Windows (where the production deployment runs). No server process is needed; concurrent writes are serialized by a file lock (`_FileLock`). The schema includes a `source` column to track which client wrote each memory. In production, 15 distinct agent clients have written to the same physical database over 26 days.

### 3.5 Threat Model

The hash-chained audit log addresses a specific and bounded adversary: an attacker with *read-only* access to the database (e.g., a rogue co-resident process that can inspect but not modify the file), or an attacker who can modify rows but is not aware of the anchoring mechanism and therefore does not recompute the chain. We explicitly do *not* claim protection against an adversary with full write access who understands the chain and recomputes anchors, or who deletes the `audit_anchors` table entirely. Defending against such an adversary requires either an external checkpointing service (introducing a trusted third party) or hardware-backed attestation (TPM), both of which conflict with our local-first design goal. We view the hash chain as a deterrent and an integrity *detector* for realistic semi-trusted scenarios, not as a cryptographic defense against a full-write adversary. This limitation is stated in the abstract and revisited in Section 6.

### 3.6 Schema-Grounded Retrieval (SGM)

At write time, a predicate extractor parses each memory and stores (entity, attribute) pairs in a `fact_entities` table. At query time, a lightweight classifier routes questions to either the embedding path (open-ended) or the SQL path (structured lookups such as counting or entity queries). On Chinese-language synthetic data, this SQL path achieves 100% recall (25/25) on entity-aggregation queries versus 62% for embedding search. However, on English-language benchmark data (LongMemEval), the regex-based Chinese predicate extractor produces zero (entity, attribute) pairs, so the SQL path does not activate and results are identical to pure embedding retrieval. We report this as a negative result with a concrete remediation path (LLM-based extraction) in Section 6.

### 3.7 Cross-System Retrieval Context

For context, we report the LongMemEval-500 accuracy of three systems evaluated on the same machine, same benchmark, and same judge configuration:

We report these numbers for context only: our contribution is not retrieval accuracy but the governance and auditability layer, which none of the three systems provide. For state-of-the-art retrieval accuracy on LongMemEval, we refer readers to Memanto (Abtahi et al., 2026) (89.8%), MemStrata (Yadav, 2026) (95%), and xmemory (Petrov et al., 2026) (97.1% F1), all of which we do not attempt to replicate in this paper.

Table 1: LongMemEval-500 strict matching accuracy across three systems (same hardware, same judge). Third-party systems evaluated with their default configurations; MemTether numbers from our production 500-question run (2026-10-05).

| System | Type | Strict | Judge |
|---|---|---|---|
| LangMem | library (LangGraph) | 21.8% (109/500) | 6.6% (33/498) |
| MemTether | local-first governance | 43.1% (135/313)[a] | 48.5% (228/470) |
| mem0 | hosted platform | 60.2% (168/279)[a] | 65.2% |

[a]Strict matching applies only to entity-type gold answers (313/500 for MemTether, 279/500 for mem0).

# 4 Tamper-Detection Evaluation

We evaluate the hash chain on a copy of the production database (2,346 audit entries, 15 sources, 26 days). After anchoring, we apply six tamper scenarios and re-run `verify_chain()`.

Table 2: Tamper-detection results on 2,346 audit entries.

| Scenario | Detected | Reason |
|---|---|---|
| Edit detail field (row 10) | ✓ | hash mismatch |
| Delete row 50 | ✓ | row-count mismatch |
| Edit op field (row 100) | ✓ | hash mismatch |
| Edit agent field (row 500) | ✓ | hash mismatch |
| Delete rows 500–509 (10 rows) | ✓ | row-count mismatch |
| Insert fake row at unanchored id 10000 | × | not yet anchored |
| No tamper (control) | — | chain OK |

The hash chain detects 100% of edit and deletion attacks on anchored rows (5/5 scenarios). Insertion of a fake row beyond the anchored tail is not detected because the anchor covers only rows up to `last_log_id`; this is a known limitation of in-database anchoring without an external checkpoint. Periodic anchoring (every 10 operations in our production configuration) bounds the unanchored tail.

# 5 Production Deployment

Table 3: 26-day production deployment metrics (2026-09-13 to 2026-10-09).

| Metric | Value |
|---|---|
| Total memory rows | 2,269 |
| Active memory rows | 1,662 |
| Distinct sources (agent clients) | 15 |
| Supersession events | 347 |
| Audit log entries | 2,346 |
| Superseded facts | 383 |
| Retired facts | 224 |
| Superseded facts with `valid_to` | 256 (67% of 383) |
| Facts with non-default Q-Value ($\neq 0.5$) | 495 |
| Operations processed (cumulative) | 31K+ |
| Data loss events | 0 |

**Metric definitions.** "Total memory rows" counts all rows ever inserted in `facts` (including superseded and retired). "Active memory rows" counts rows with `status='active'`. "Supersession events" counts rows in `supersessions`. "Audit log entries" counts rows in `audit_log`. "Facts with `valid_to`" counts active or superseded rows with a non-empty `valid_to` timestamp. "Facts with non-default Q-Value" counts active rows where $q \neq 0.5$ (the initial value). "Data loss events" is a manual audit: no reported incident of a memory row disappearing or being corrupted in 26 days of production use. "Operations processed" is the sum of `audit_log` entries (writes) and acknowledged `use_count` increments (reads).

# 6 Discussion and Limitations

**Why not compete on retrieval accuracy?** Memanto (Abtahi et al., 2026) (89.8%), MemStrata (Yadav, 2026) (95%), and xmemory (Petrov et al., 2026) (97.10% F1) achieve substantially higher retrieval accuracy on LongMemEval than the embedding baseline we measure in our production configuration (43.1%, strict matching, 500-question subset; see below for caveats on cross-paper comparability). We do not claim retrieval superiority. Our contribution is the *governance and auditability layer* that none of these systems provide: tamper-evident hash chains, human adjudication, file-level multi-client sharing, and enforcement of source attribution at the write path. We view these as complementary to retrieval optimization.

**English predicate extraction.** Our SQL retrieval path (SGM) currently relies on a regex-based Chinese predicate extractor ("X de Y" patterns). On English-language data, this extractor produces no (entity, attribute) pairs, so the SQL path does not activate. The measured 100% recall on Chinese synthetic data (25/25) is real but does not transfer to English benchmarks. An LLM-based triple extractor would address this, at the cost of write-path latency; we leave this to future work.

**Audit anchoring limitation.** The hash chain detects edits and deletions of *anchored* rows. Rows appended after the last anchor are not covered until the next anchor fires (every 10 operations in our configuration). An external checkpointing service would extend detection to the unanchored tail but introduces a trust dependency; we chose to keep the system local-first.

**Single-machine scope.** MemTether is designed for multi-client deployment on a single machine via file-level pointers. Cross-machine memory sharing requires different infrastructure (e.g., mem0's hosted service) and is out of scope.

**Production window.** 26 days of production data is a bounded observation. Longitudinal claims (forgetting curves, memory quality degradation) require 6+ months.

# 7 Conclusion

We presented MemTether, a local-first agent memory system that integrates bi-temporal supersession, hash-chained tamper-evident audit, and file-level multi-client deployment in a single architecture. In 26 days of production use across 15 agent clients, the system processed over 31K recorded operations (2,346 writes tracked in the audit log plus 28,807 tracked reads) with zero data loss. A controlled tamper-detection evaluation confirmed that the hash chain detects 100% of edit and deletion attacks on anchored audit entries. We additionally reported a negative result on English-language schema-grounded retrieval, identifying the absence of a general English predicate extractor as a concrete open problem. The system is released as Apache-2.0 open source with 488 passing tests and a 9-job CI matrix.

# References

Wu, C. et al. LongMemEval: Benchmarking Long-Term Memory in LLM Agents. *arXiv:2410.10813*, 2024.

Chhikara, P. et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, 2025.

Rasmussen, R. et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956*, 2025.

Packwood, S. et al. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023. Letta, 2025.

Liu, H. et al. LangMem: Memory Management for Language Agents. *LangChain*, 2025.

Wang, L. Less Context, More Accuracy: A Bi-Temporal Memory Model for AI Agents. *arXiv:2606.09900*, 2026.

Kim, J. et al. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. *NeurIPS 2025 Spotlight*, 2025.

Petrov, A., Gusak, A., Mukha, D., and Korolev, D. From Unstructured Recall to Schema-Grounded Memory: Reliable AI Memory via Iterative, Schema-Aware Extraction. *arXiv:2604.27906v3*, 2026.

Abtahi, S. M., Rahnema, R., Patel, H., Patel, N., Fekri, M., and Khani, T. Memanto: Typed Semantic Memory with Information-Theoretic Retrieval for Long-Horizon Agents. *arXiv:2604.22085*, 2026.

Yadav, N. Temporal Validity in Retrieval Memory: Eliminating Stale-Fact Errors for AI Agents over Evolving Knowledge. *arXiv:2606.26511*, 2026.

Anonymous. A2A-ForensicTrace: Offline Verification of Tamper-Evident A2A Runtime Evidence. *arXiv:2609.33924*, 2026.

Anonymous. Proof-of-Execution Memory: Defending LLM Agents Against Forged-Reasoning Attacks by Verifying What Agents Do. *arXiv:2608.16032*, 2026.

Anonymous. MemMark: State-Evolution Attribution Watermarking for Agent Long-Term Memory Systems. *arXiv:2605.25002*, 2026.

European Parliament and Council. Regulation (EU) 2024/1689 (Artificial Intelligence Act). *Official Journal of the European Union*, 2024.